Name of Student: Husam Nour	**PRN:25070122086**
Semester: III	**Year : AY 2026-27**
Subject: Programming with Java	
ASSIGNMENT No : 30	
TITLE : Write a JSP application to demonstrate CRUD operations.	

Problem Statement

Write a JSP application to demonstrate CRUD operations.

Learning Objectives

To understand and implement CRUD operations (Create, Read, Update, and Delete) in a JSP application for managing database records.

Theory

CRUD stands for Create, Read, Update, and Delete, which represent the four basic database operations performed in web applications. JSP works together with Servlets and JDBC to implement these operations. Users interact with HTML forms created using JSP, while JDBC executes SQL statements to manipulate database records. INSERT adds new records, SELECT retrieves information, UPDATE modifies existing data, and DELETE removes unwanted records. Proper validation, exception handling, and session management improve application reliability and security. JSP CRUD applications are widely used in employee management systems, online shopping portals, hospital management systems, banking applications, and other enterprise software.

1. **Create a JSP-based applica on to add, update, delete, and view employee details**

Programme:

```
<%@ page import="java.sql.*" %>

<!DOCTYPE html>

<html>

<head>

  <title>Employee CRUD</title>

</head>

<body>

  <h2>Employee Management System</h2>

  <form method="post">

    Employee ID:
```

```
    <input type="number" name="id" required><br><br>
    Name:
    <input type="text" name="name"><br><br>
    Department:
    <input type="text" name="department"><br><br>
    Salary:
    <input type="number" step="0.01" name="salary"><br><br>
    <input type="submit" name="action" value="Add">
    <input type="submit" name="action" value="Update">
    <input type="submit" name="action" value="Delete">
    <input type="submit" name="action" value="View">
  </form>
  <hr>
<%
  String url = "jdbc:mysql://localhost:3306/jspcrud";
  String user = "root";
  String password = "Husam@123";
  String action = request.getParameter("action"); try
  {
    Class.forName("com.mysql.cj.jdbc.Driver");
    Connection con =
        DriverManager.getConnection(url, user, password); if
    ("Add".equals(action)) { int id =
    Integer.parseInt(request.getParameter("id")); String name =
    request.getParameter("name"); String department =
    request.getParameter("department"); double salary =
        Double.parseDouble(request.getParameter("salary"));
      PreparedStatement ps = con.prepareStatement(
        "INSERT INTO employee VALUES (?, ?, ?, ?)"
```

```
    );
    ps.setInt(1, id); ps.setString(2, name); ps.setString(3,
    department);          ps.setDouble(4,          salary);
    ps.executeUpdate();       out.println("<p>Employee
    added successfully.</p>");
} else if ("Update".equals(action)) { int id =
   Integer.parseInt(request.getParameter("id")); String
   name = request.getParameter("name");
   String department = request.getParameter("department"); double
   salary =
        Double.parseDouble(request.getParameter("salary"));
   PreparedStatement ps = con.prepareStatement(
     "UPDATE employee SET name=?, department=?, salary=? WHERE id=?"
   );
   ps.setString(1, name); ps.setString(2, department);
   ps.setDouble(3,       salary);       ps.setInt(4,       id);
   ps.executeUpdate();          out.println("<p>Employee
   updated successfully.</p>");
} else if ("Delete".equals(action)) { int id =
   Integer.parseInt(request.getParameter("id"));
   PreparedStatement ps = con.prepareStatement(
     "DELETE FROM employee WHERE id=?"
   );
   ps.setInt(1, id);
   ps.executeUpdate();
   out.println("<p>Employee deleted
   successfully.</p>");
```

```
    } else if ("View".equals(action))
        { Statement st = con.createStatement(); ResultSet
        rs = st.executeQuery(
           "SELECT * FROM employee"
        );
        out.println("<h3>Employee Details</h3>");
        out.println("<table border='1' cellpadding='8'>");
        out.println("<tr>"); out.println("<th>ID</th>");
        out.println("<th>Name</th>");
        out.println("<th>Department</th>");
        out.println("<th>Salary</th>"); out.println("</ tr>"); while
        (rs.next()) { out.println("<tr>"); out.println("<td>" +
        rs.getInt("id") + "</td>"); out.println("<td>" +
        rs.getString("name") + "</td>"); out.println("<td>" +
        rs.getString("department") + "</td>"); out.println("<td>" +
        rs.getDouble("salary") + "</td>"); out.println("</tr>");
        }
        out.println("</table>");
    }
    con.close();
  } catch (Exception e) { out.println("<p>Error: " +
    e.getMessage() + "</p>");
  }
%>
</body>
</html>
```

Output: http://localhost:8080/StudentApp/

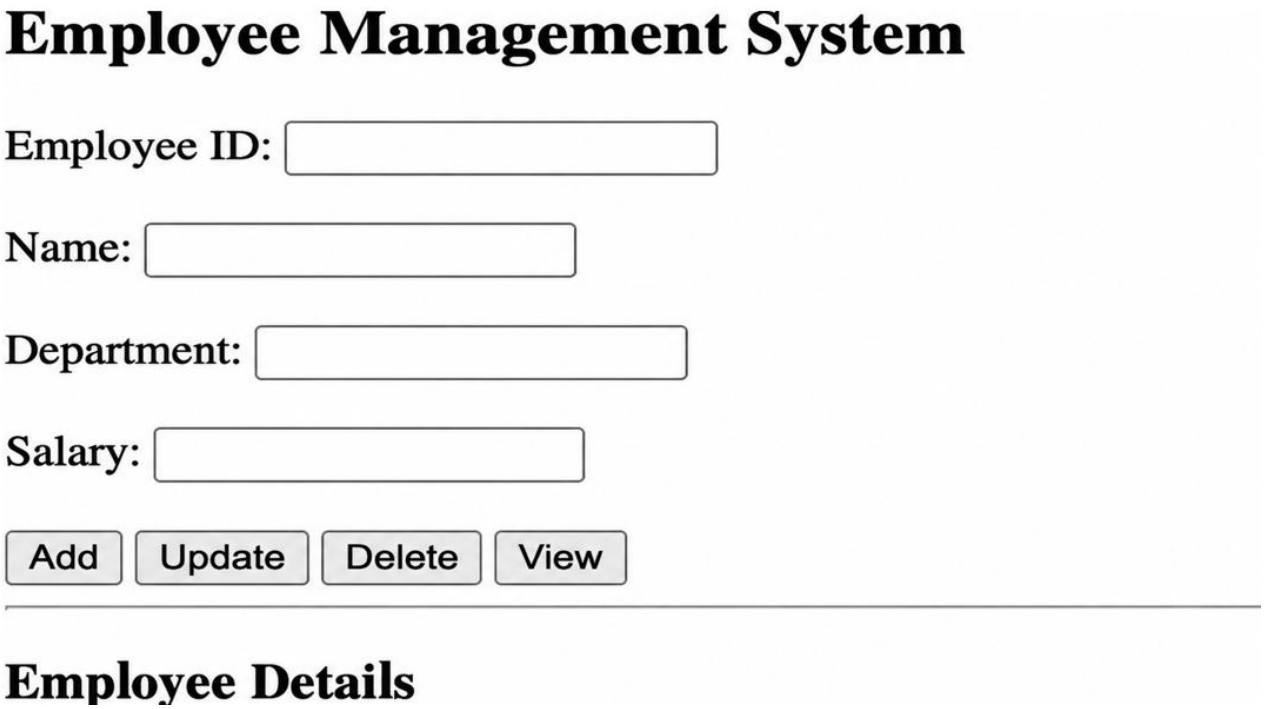
patient.jsp

```
C:\Users\husam\OneDrive\Documents\Personal\CMakeLists.txt> wsl -l -v -q

C:\Users\husam\OneDrive\Documents\Personal\CMakeLists.txt> wsl -d Ubuntu-22.04 -e dpkg -l | grep -i cmake
ii  cmake-data        3.22.1-1ubuntu1.22.04.1   all          CMake data files
ii  cmake             3.22.1-1ubuntu1.22.04.1   amd64        cross-platform, open-source make system
ii  cmake-curses-gui  3.22.1-1ubuntu1.22.04.1   amd64        curses-based user interface for CMake
ii  libcmake3         3.22.1-1ubuntu1.22.04.1   amd64        CMake library
ii  cmake-gui         3.22.1-1ubuntu1.22.04.1   amd64        graphical user interface for CMake

C:\Users\husam\OneDrive\Documents\Personal\CMakeLists.txt> wsl -d Ubuntu-20.04 -e dpkg -l | grep -i cmake
ii  cmake-data        3.16.3-1ubuntu1.20.04.1   all          CMake data files
ii  cmake             3.16.3-1ubuntu1.20.04.1   amd64        cross-platform, open-source make system
ii  cmake-curses-gui  3.16.3-1ubuntu1.20.04.1   amd64        curses-based user interface for CMake
ii  libcmake3         3.16.3-1ubuntu1.20.04.1   amd64        CMake library
ii  cmake-gui         3.16.3-1ubuntu1.20.04.1   amd64        graphical user interface for CMake

C:\Users\husam\OneDrive\Documents\Personal\CMakeLists.txt> wsl -d Ubuntu -e dpkg -l | grep -i cmake
ii  cmake-data        3.28.3-1ubuntu1           all          CMake data files
ii  cmake             3.28.3-1ubuntu1           amd64        cross-platform, open-source make system
ii  cmake-curses-gui  3.28.3-1ubuntu1           amd64        curses-based user interface for CMake
ii  libcmake3         3.28.3-1ubuntu1           amd64        CMake library
ii  cmake-gui         3.28.3-1ubuntu1           amd64        graphical user interface for CMake
```



2. Create a JSP-based application to manage patient records using CRUD operations. Include fields such as patient ID, name, age, disease, and treatment status.

Programme:

<%@ page import="java.sql.*" %>

<!DOCTYPE html>

<html>

<head>

```
    <title>Patient CRUD</title>
</head>
<body>
    <h2>Patient Management System</h2>
    <form method="post">
        Patient ID:
        <input type="number" name="patient_id" required><br><br>
        Name:
        <input type="text" name="name"><br><br> Age:
        <input type="number" name="age"><br><br> Disease:
        <input type="text" name="disease"><br><br>
        Treatment Status:
        <input type="text" name="treatment_status"><br><br>
        <input type="submit" name="action" value="Add">
        <input type="submit" name="action" value="Update">
        <input type="submit" name="action" value="Delete">
        <input type="submit" name="action" value="View">
    </form>
    <hr>
<%
    String url = "jdbc:mysql://localhost:3306/jspcrud";
    String user = "root";
    String password = "Husam@123";
    String action = request.getParameter("action"); try
    {
        Class.forName("com.mysql.cj.jdbc.Driver");
        Connection con =
                DriverManager.getConnection(url, user, password);
```

```
if ("Add".equals(action))
    { int patientId =
          Integer.parseInt(request.getParameter("patient_id")
    ); String name = request.getParameter("name"); int age =
         Integer.parseInt(request.getParameter("age"));
    String                disease                =
          request.getParameter("disease");
    String                      status                      =
          request.getParameter("treatment_status");
    PreparedStatement ps = con.prepareStatement(
       "INSERT INTO patient VALUES (?, ?, ?, ?, ?)"
    );
    ps.setInt(1,  patientId);  ps.setString(2,  name);
    ps.setInt(3,     age);     ps.setString(4,     disease);
    ps.setString(5,     status);     ps.executeUpdate();
    out.println("<p>Patient                    added
    successfully.</p>");
} else if ("Update".equals(action))
    { int patientId =
        Integer.parseInt(request.getParameter("patient_id"));
    String name =
    request.getParameter("name"); int age
    =
          Integer.parseInt(request.getParameter("age"));
    String disease =
          request.getParameter("disease");
          String status =
```

```
            request.getParameter("treatment_status");
        PreparedStatement ps = con.prepareStatement(
          "UPDATE patient SET name=?, age=?, disease=?, treatment_status=? WHERE
patient_id=?"
        );
        ps.setString(1,      name);      ps.setInt(2,      age);
        ps.setString(3,   disease);   ps.setString(4,   status);
        ps.setInt(5,      patientId);      ps.executeUpdate();
        out.println("<p>Patient                          updated
        successfully.</p>");
    } else if ("Delete".equals(action))
        { int patientId =
            Integer.parseInt(request.getParameter("patient_id"));
        PreparedStatement ps = con.prepareStatement(
          "DELETE FROM patient WHERE patient_id=?"
        );
        ps.setInt(1,      patientId);      ps.executeUpdate();
        out.println("<p>Patient                          deleted
        successfully.</p>");
    } else if ("View".equals(action))
        { Statement st = con.createStatement();
        ResultSet rs = st.executeQuery(
          "SELECT * FROM patient"
        );
        out.println("<h3>Patient             Details</h3>");
        out.println("<table                          border='1'
        cellpadding='8'>"); out.println("<tr>");

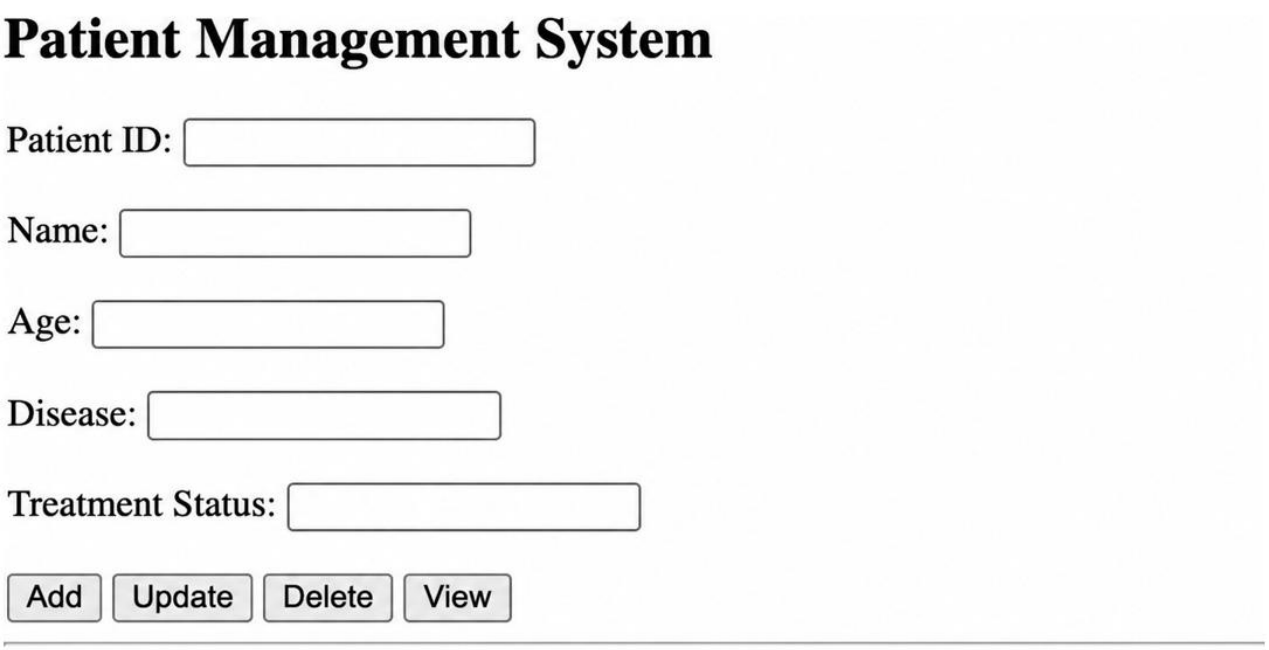
        out.println("<th>Patient ID</th>");
```

```
        out.println("<th>Name</th>");
        out.println("<th>Age</                    th>");
        out.println("<th>Disease</th>");
        out.println("<th>Treatment          Status</th>");
        out.println("</tr>");      while      (rs.next())      {
        out.println("<tr>"); out.println(
            "<td>" + rs.getInt("patient_id") + "</td>"
          );
          out.println(
            "<td>" + rs.getString("name") + "</td>"
          );
          out.println(
            "<td>" + rs.getInt("age") + "</td>"
          );
          out.println(
            "<td>" + rs.getString("disease") + "</td>"
          );
          out.println(
            "<td>" + rs.getString("treatment_status") + "</td>"
          );
          out.println("</tr>");
        }
        out.println("</table>");
    }
    con.close();
} catch (Exception e)
    { out.println(
        "<p>Error: " + e.getMessage() + "</p>"
    );
```

```
    }
%>
</body>
</html>
```

Output:

http://localhost:8080/StudentApp/patient.jsp



Conclusion

Thus, the JSP application for CRUD operations was successfully implemented. The application was used to add, view, update, and delete patient records using JSP, JDBC, and MySQL. This experiment helped us understand how a web application can interact with a database to manage records efficiently.